

# PyTorch\_conv1d 001

Testing pytorch conv1d for DSP convolution

```
import numpy as np
import scipy.signal as ss
import matplotlib.pyplot as plt
import IPython.display as ipd
from scipy.io import wavfile
import torch
import torch.nn.functional as F

%matplotlib widget
fsz = (7,4)
fsz2 = (fsz[0], 1.5*fsz[1]/2)

# 1. Define 1D signal and kernel
# Signal (X): Length M = 7
signal = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0, 3.0, 4.0])
# Kernel (H): Length L = 5
kernel = torch.tensor([0.5, 1.0, -0.5, 2.0, -0.8])

# Visualize tensor attributes of signal tensor
print(f"Shape:          {signal.shape}")          # Output: torch.Size([7])
print(f"Data Type:     {signal.dtype}")          # Output: torch.float32
print(f"Device:         {signal.device}")          # Output: cpu
print(f"Layout:          {signal.layout}")          # Output: torch.strided
print(f"Requires Grad:{signal.requires_grad}")    # Output: False
print(f"Current Grad: {signal.grad}")             # Output: None

Shape:          torch.Size([7])
Data Type:      torch.float32
Device:         cpu
Layout:         torch.strided
Requires Grad:False
Current Grad: None

# 2. Reshape for PyTorch conv1d: (batch_size, num_channels, signal_length)
# Both require 3D input shape: [1, 1, M] and [1, 1, L]
X = signal.view(1, 1, -1)
H = kernel.view(1, 1, -1)

# Visualize tensor attributes of X tensor
print(f"Shape:          {X.shape}")          # Output: torch.Size([1, 1, 7])
print(f"Data Type:     {X.dtype}")          # Output: torch.float32
print(f"Device:         {X.device}")          # Output: cpu
print(f"Layout:          {X.layout}")          # Output: torch.strided
print(f"Requires Grad:{X.requires_grad}")    # Output: False
print(f"Current Grad: {X.grad}")             # Output: None
```

```
Shape:      torch.Size([1, 1, 7])
Data Type:  torch.float32
Device:     cpu
Layout:     torch.strided
Requires Grad: False
Current Grad: None
```

```
# 3. DSP Note: conv1d does cross-correlation.
# For true mathematical convolution, flip the kernel along the time axis.
H_flipped = torch.flip(H, dims=[-1])
```

```
# Visualize H and H_Flipped
torch.set_printoptions(profile="full")
print(f"H:      {H}")
print(f"H_flipped: {H_flipped}")
torch.set_printoptions(profile="default")
print(f"H:      {H}")
print(f"H_flipped: {H_flipped}")
# Select elements
print(f"H:      {H[0, 0, :]}")
print(f"H_flipped: {H_flipped[0, 0, :]}")
# Dimensions
print(f"H.shape[0]: {H.shape[0]}")
print(f"H.shape[1]: {H.shape[1]}")
print(f"H.shape[2]: {H.shape[2]}")
```

```
H:      tensor([[[ 0.5000,  1.0000, -0.5000,  2.0000, -0.8000]]])
H_flipped: tensor([[[ -0.8000,  2.0000, -0.5000,  1.0000,  0.5000]]])
H:      tensor([[[ 0.5000,  1.0000, -0.5000,  2.0000, -0.8000]]])
H_flipped: tensor([[[ -0.8000,  2.0000, -0.5000,  1.0000,  0.5000]]])
H:      tensor([ 0.5000,  1.0000, -0.5000,  2.0000, -0.8000])
H_flipped: tensor([ -0.8000,  2.0000, -0.5000,  1.0000,  0.5000])
H.shape[0]: 1
H.shape[1]: 1
H.shape[2]: 5
```

```
# 4. Set padding
# For 'full' linear convolution output size (M + L - 1), pad X by (L - 1)
padding_val = H.shape[-1] - 1
# pad = (left, right, top, bottom)
#X_padded = F.pad(X, pad=(padding_val, 0, 0, 0), mode='constant', value=0)
# left-padding by padding_val zeros
# left/right-padding by padding_val zeros
X_padded = F.pad(X, pad=(padding_val, padding_val, 0, 0), mode='constant',
value=0)
```

```
# Visualize X and X_padded
print(f"X:      {X}")
print(f"X_padded: {X_padded}")
```

```
X:          tensor([[[[-2., -1.,  0.,  1.,  2.,  3.,  4.]])])
X_padded:  tensor([[[[ 0.,  0.,  0.,  0., -2., -1.,  0.,  1.,  2.,  3.,  4.,
 0.,  0.,  0.,
 0.]])])
```

*# 5. Execute convolution*

*# We use functional conv1d so we can pass our custom weights directly.*

```
Y_out = F.conv1d(X_padded, H_flipped)
```

*# Visualize Y\_conv1d*

```
print(f"Y_out:          {Y_out}")
print(f"Y_out[0, 0, :]: {Y_out[0, 0, :]}")
```

```
Y_out:          tensor([[[[-1.0000, -2.5000,  0.0000, -3.0000,  1.6000,
 3.8000,  6.0000,
          5.7000,  2.4000,  5.6000, -3.2000]])])
Y_out[0, 0, :]: tensor([-1.0000, -2.5000,  0.0000, -3.0000,  1.6000,
 3.8000,  6.0000,  5.7000,
          2.4000,  5.6000, -3.2000])
```

*# Print DSP convolution*

```
Y_DSP = np.convolve(signal.numpy(), kernel.numpy(), mode='full')
print(f"Y_DSP: {Y_DSP}")
```

```
Y_DSP: [-1.  -2.5  0.  -3.   1.6  3.8  6.   5.7  2.4  5.6 -3.2]
```

*# 6. Clean up output shape back to a 1D array*

```
Y_conv1d = Y_out.squeeze()
```

```
print("Signal:      ", signal.tolist())
print("Kernel:      ", kernel.tolist())
print("Y_conv1d:     ", Y_conv1d.tolist())
print("Y_conv1d:     ", Y_conv1d.numpy())
print("Y_DSP:        ", Y_DSP)
print("Difference:   ", np.max(np.abs((Y_DSP-Y_conv1d.tolist()))))
```

```
Signal:      [-2.0, -1.0, 0.0, 1.0, 2.0, 3.0, 4.0]
Kernel:      [0.5, 1.0, -0.5, 2.0, -0.8000000011920929]
Y_conv1d:     [-1.0, -2.5, 0.0, -3.0, 1.6000000023841858, 3.7999999952316284,
 6.0, 5.699999809265137, 2.40000000953674316, 5.599999904632568, -
 3.2000000047683716]
Y_conv1d:     [-1.  -2.5  0.  -3.   1.6  3.8  6.   5.7  2.4  5.6 -3.2]
Y_DSP:        [-1.  -2.5  0.  -3.   1.6  3.8  6.   5.7  2.4  5.6 -3.2]
Difference:    0.0
```